Digital Logic and Computer Design (DLCD)

Question Bank – 5, 8 and 10 Mark Questions with Answers

Based on the Unit PPTs: Introduction, Assembly Language, Machine Language, Programming, Addressing Modes, Parallelism, Sequential Building Blocks, Memory Arrays

How to use: every answer has exactly as many numbered points as marks – **5 marks = 5 points, 8 marks = 8 points, 10 marks = 10 points** (1 point = 1 mark). Each deck has 2 questions of 5 marks, 2 of 8 marks and 2 of 10 marks.

1. Introduction (Computer Architecture)

Q1. [5 Marks] Define computer architecture. State what it specifies and why it is important to study it.

Answer (5 points):

1. **Definition:** Computer architecture is the programmer's view of a computer; it defines how software communicates with hardware.
2. **Instruction Set Architecture (ISA):** it specifies the set of instructions the processor understands.
3. **Operand locations:** it specifies where operands are kept – registers and memory.
4. **Data representation and instruction formats:** it specifies how data and instructions are encoded in bits.
5. **Importance:** it bridges the gap between hardware and software, decides how programs are executed, helps design efficient, compatible, high-performance systems and is the foundation of processor organisation and programming.

Q2. [5 Marks] Differentiate between assembly language and machine language with an example. Who developed MIPS?

Answer (5 points):

1. **Instructions** are the "words" of a computer's language; the collection of instructions is the instruction set.
2. **Assembly language** is the human-readable form of instructions using mnemonics, e.g. add $s0, $s1, $s2.
3. **Machine language** is the computer-readable form made of 1's and 0's (32-bit in MIPS), e.g. the same instruction is 0x02328020.

```
add $s0, $s1, $s2
000000 10001 10010 10000 00000 100000  = 0x02328020
```

4. **Translation:** assembly is translated into machine code by an assembler; the processor executes only machine code, while assembly is easier to write, read and debug.
5. **MIPS** was developed by John Hennessy and colleagues at Stanford in the 1980s.

Q3. [8 Marks] Explain the statement "Instruction sets are like dialects of a language". Name some architectures and explain how complex applications finally run on the processor.

Answer (8 points):

1. Many architectures exist, e.g. **x86, MIPS, SPARC and PowerPC**.
2. Each architecture defines its **own instruction set**.
3. **Dialect analogy:** dialects of one language share the same structure but differ in vocabulary; likewise instruction sets share the same basic ideas (arithmetic, logic, load/store, branch) but differ in names and formats.
4. Hence, **once one architecture is learnt, learning another is fairly straightforward**.
5. **Instructions are the words** of the computer's language.

6. Even complex applications are **compiled down to simple instructions** such as add, subtract and jump.
7. The processor fetches and executes these simple instructions one after another.
8. The ISA is therefore the interface between software and hardware: a program must be compiled for the ISA of the machine on which it runs.

Q4. [8 Marks] Describe the MIPS architecture – its full form, origin, philosophy and applications.

Answer (8 points):

1. **Full form:** Microprocessor without Interlocked Pipelined Stages.
2. MIPS is a family of **Reduced Instruction Set Computer (RISC)** instruction set architectures.
3. It was developed by **John Hennessy and colleagues at Stanford in the 1980s**.
4. It was commercialised by MIPS Computer Systems (now MIPS Technologies), United States.
5. It is used in commercial systems such as **Silicon Graphics, Nintendo and Cisco**.
6. **Philosophy:** a small set of simple, regular instructions with 32-bit data and 32-bit instructions that can be decoded and executed quickly.
7. It contrasts with CISC architectures like Intel x86, where one instruction performs a more complex task.
8. Once MIPS is learnt, other architectures are easy to learn.

Q5. [10 Marks] Explain how a high-level language statement is finally executed by the hardware. Illustrate the levels of abstraction using a = b + c.

Answer (10 points):

1. **High-level language** (C, Java, Python) works at a high abstraction level using if/else, loops, arrays and function calls.
2. A **compiler** converts high-level code into simple instructions of the target ISA.
3. **Assembly language** represents each instruction with a human-readable mnemonic.
4. An **assembler** converts assembly into **machine language**, a 32-bit binary pattern stored in memory.
5. **Example in C:** a = b + c; → MIPS assembly: add a, b, c.

```
C:    a = b + c;
MIPS: add a, b, c
```

6. In the assembly instruction, **add** is the mnemonic (operation), **b and c** are the source operands and **a** is the destination operand.
7. With real registers (a=$s0, b=$s1, c=$s2) the instruction becomes **add $s0, $s1, $s2**.
8. Its **machine code** is 000000 10001 10010 10000 00000 100000 = 0x02328020.

```
op=000000 rs=10001($s1) rt=10010($s2) rd=10000($s0) shamt=00000
funct=100000(add)
```

9. The CPU **fetches** this 32-bit word from memory and **decodes** the opcode/funct fields.
10. It reads $s1 and $s2, the **ALU adds** them and the result is **written back to $s0**.

Q6. [10 Marks] Explain the role of the Instruction Set Architecture as the interface between software and hardware. Compare RISC (MIPS) and CISC (x86) architectures.

Answer (10 points):

1. **ISA** is the set of instructions, operand locations, data representation and instruction formats that a processor supports.
2. It is the **programmer's view** of the computer and hides how the hardware is built internally.
3. Software compiled for an ISA can run on **any hardware that implements that ISA**, which gives compatibility.

4. Hardware designers may implement the same ISA in different ways to trade off performance, cost and power.
5. Examples of ISAs: **x86, MIPS, SPARC, PowerPC**; each has its own instruction set.
6. **RISC (MIPS):** a small set of simple, regular instructions that are decoded and executed quickly.
7. In RISC, complex operations are built by **combining several simple instructions**.
8. RISC gives simpler hardware, faster execution and lower power consumption.
9. **CISC (x86):** individual instructions perform more complex tasks, so fewer instructions are needed per program but hardware is more complex.
10. This is the design principle **"Good design demands good compromises"** – no design optimises everything.

Feature	RISC (MIPS)	CISC (x86)
Instructions	Simple	Complex
Hardware	Simpler	More complex
Instructions per program	More	Fewer
Execution speed	Faster per instruction	Slower per instruction

2. Assembly Language (MIPS)

Q1. [5 Marks] State and explain the four design principles of computer architecture.

Answer (5 points):

1. **Simplicity favors regularity:** uniform instruction formats (e.g. add/sub both use two sources and one destination) simplify encoding, decoding and hardware.
2. **Make the common case fast:** frequently executed operations are optimised; MIPS has a small set of simple instructions (RISC) and builds complex operations from them.
3. **Smaller is faster:** small structures such as the 32-register file have lower access time than large memory.
4. **Good design demands good compromises:** performance, cost, power and complexity must be balanced (e.g. RISC vs CISC).
5. **Example:** MIPS applies all four – regular formats, simple common instructions, only 32 registers and a RISC compromise.

Q2. [5 Marks] What are the three operand locations in MIPS? Explain with the example addi $t0, $t1, 10.

Answer (5 points):

1. An **operand** is the data used by an instruction; it can be stored in different locations.
2. **Registers:** high-speed storage inside the CPU for frequently used data (32 registers).
3. **Memory:** large storage for programs and data that do not fit in registers; accessed using lw and sw.
4. **Constants (immediates):** fixed values encoded directly in the instruction (16-bit signed).
5. **Example:** in addi $t0, $t1, 10, $t0 is the destination register, $t1 the source register and 10 the immediate; $t0 = $t1 + 10.

Q3. [8 Marks] Describe the MIPS register set. Why does MIPS have only 32 registers? Explain the usage of the important registers.

Answer (8 points):

1. MIPS has **32 general-purpose registers, each 32 bits wide**, so it is called a 32-bit architecture.
2. Every register name is prefixed with **$** (e.g. $0 is read "dollar zero").
3. **Why only 32 – "Smaller is faster":** a small register file has lower access time and balances performance and hardware complexity.
4. **$0** always holds the constant 0 and cannot be modified.
5. **$s0–$s7 (saved):** variables whose values must be preserved across function calls.
6. **$t0–$t9 (temporaries):** intermediate results; need not be preserved.
7. **$a0–$a3** pass function arguments and **$v0–$v1** hold return values.
8. Special registers: **$at** (assembler temporary), **$k0–$k1** (OS), **$gp** (global pointer), **$sp** (stack pointer), **$fp** (frame pointer) and **$ra** (return address).

Name	No.	Usage
$0	0	constant 0
$at	1	assembler temporary
$v0–$v1	2–3	return values
$a0–$a3	4–7	arguments
$t0–$t7	8–15	temporaries
$s0–$s7	16–23	saved variables
$t8–$t9	24–25	more temporaries
$k0–$k1	26–27	OS temporaries

Name	No.	Usage
$gp $sp $fp $ra	28 29 30 31	global, stack, frame pointer, return address

Q4. [8 Marks] Compare word-addressable and byte-addressable memory. Explain lw and sw with effective address calculation.

Answer (8 points):

1. **Word-addressable memory:** each address refers to one complete word (32 bits = 4 bytes); individual bytes cannot be addressed directly.
2. **Byte-addressable memory:** each address refers to exactly one byte (8 bits).
3. A 32-bit word occupies **1 address** in word-addressable memory but **4 consecutive addresses** in byte-addressable memory.
4. Word addresses increase by 1 word (0,1,2,3); byte addresses increase by 1 byte, so consecutive words start at **0, 4, 8, 12**.
5. Byte-addressable memory is more flexible because individual bytes can also be accessed; **MIPS uses byte-addressable memory**.
6. **lw (load word)** reads memory into a register; **sw (store word)** writes a register to memory: lw dest, offset(base).
7. **Effective Address = Base Register + Offset**; in MIPS the offset is in bytes, not words.
8. **Examples:** lw $s3, 4($0) → EA = 0 + 4 = 4, bytes 4–7 are loaded into $s3; lw $s0, 5($t1) with $t1 = 20 → EA = 25; sw $t4, 0x7($0) → value of $t4 written to location 7.

```
lw $s3, 4($0)    # EA = 0 + 4 = 4 -> bytes 4,5,6,7
lw $s0, 5($t1)   # $t1=20 -> EA = 25
sw $t4, 0x7($0)  # write $t4 at EA = 7
```

Q5. [10 Marks] Translate the following into MIPS assembly: (a) a = b – c (b) f = (g + h) – (i + j) (c) a = a + 4; b = a – 12. Assume a–c in $s0–$s2 and f–j in $s3–$s7. Explain why there is no subi.

Answer (10 points):

1. **Add format:** add a, b, c – add is the mnemonic, b and c are sources, a is the destination.
2. **Sub format:** sub a, b, c performs a = b – c; it follows the same regular format.
3. **(a)** a = b – c:

```
# $s0=a $s1=b $s2=c $s3=f $s4=g $s5=h $s6=i $s7=j
sub $s0, $s1, $s2     # a = b - c
```

4. Each MIPS instruction has only **two sources and one destination**, so longer expressions need **temporary registers** for intermediate results.
5. **(b)** f = (g+h) – (i+j) needs four instructions:

```
add $t0, $s4, $s5     # t0 = g + h
add $t1, $s6, $s7     # t1 = i + j
sub $s3, $t0, $t1     # f = t0 - t1
```

6. For a = b + c – d, two instructions are used: add t, b, c then sub a, t, d (t = temporary).
7. **Immediate operands** are constants embedded in the instruction (16-bit two's complement), avoiding a memory access or extra register.
8. **(c)** a = a + 4 and b = a – 12:

```
addi $s0, $s0, 4      # a = a + 4
addi $s1, $s0, -12    # b = a - 12
```

9. **Why no subi?** Subtraction of a constant is done by adding a negative immediate, so a separate instruction is unnecessary (regularity; make the common case fast).
10. **Think-about-it:** a left shift by 3 (×8) can be done with only add: add $t0,$s0,$s0 repeated three times, using just one register.

Q6. [10 Marks] Explain big-endian and little-endian byte ordering. If $t0 = 0x23456789, find $s0 after sw $t0, 0($0) and lb $s0, 1($0) in both systems. How can a program detect the endianness of a machine?

Answer (10 points):

1. A word has several bytes, so a rule is needed for **how bytes are numbered within a word**.
2. **Big-endian:** numbering starts at the big (most significant) end – the MSB is stored at the lowest byte address.
3. **Little-endian:** numbering starts at the little (least significant) end – the LSB is stored at the lowest byte address.
4. The **word address is the same** in both systems.
5. The names come from Gulliver's Travels (eggs broken at the big or little end); the choice does not matter **except when two systems share data**.
6. For the word 0x23456789 stored at address 0, the byte layout is:

Byte address	**0**	**1**	**2**	**3**
Big-endian	23	45	67	89
Little-endian	89	67	45	23

7. **sw $t0, 0($0)** stores the word at address 0; **lb $s0, 1($0)** loads the single byte at address 1.
8. **Big-endian:** byte at address 1 is 0x45, so **$s0 = 0x00000045**.
9. **Little-endian:** byte at address 1 is 0x67, so **$s0 = 0x00000067**.
10. **Detecting endianness:** store 1 as a word and read the byte at address 0 – if it is 1 the machine is little-endian, if 0 it is big-endian. (Without lb/sb, the bytes must be separated by other means such as shifts and masks.)

```
addi $t0, $0, 1
sw   $t0, 0($0)
lb   $s0, 0($0)    # 1 -> little-endian, 0 -> big-endian
```

3. Machine Language (MIPS)

Q1. [5 Marks] Explain the three MIPS instruction formats (R, I and J) with their field sizes.

Answer (5 points):

1. All MIPS instructions are **32 bits** long (simplicity favors regularity) and come in three formats.
2. **R-type** (register operands): op(6) | rs(5) | rt(5) | rd(5) | shamt(5) | funct(6); op = 0 and funct selects the exact operation.
3. In R-type, **rs and rt are sources, rd is the destination** and shamt is the shift amount (otherwise 0).
4. **I-type** (immediate): op(6) | rs(5) | rt(5) | imm(16); used for immediates, load/store and branches; the opcode alone decides the operation.
5. **J-type** (jump): op(6) | addr(26); used for j and jal. Decode order: opcode → format → field meaning → assembly instruction.

Q2. [5 Marks] Explain the logical instructions and, or, xor, nor and their immediate forms with applications.

Answer (5 points):

1. **and** is used for masking bits, e.g. 0xF234012F AND 0x000000FF = 0x0000002F.
2. **or** is used for combining bit fields, e.g. 0xF2340000 OR 0x000012BC = 0xF23412BC.
3. **xor** shows differences between bits.
4. **nor** is OR followed by inversion; A NOR $0 = NOT A, so MIPS has no separate NOT.
5. **andi, ori, xori** use a 16-bit immediate that is **zero-extended**; nori is not needed because ori followed by nor with $0 gives the same result.

Q3. [8 Marks] Explain the shift instructions sll, srl, sra and the variable shifts sllv, srlv, srav with examples.

Answer (8 points):

1. Shift instructions move the bits of a register left or right by **0 to 31** positions.
2. **sll (shift left logical):** fills the least significant bits with 0; shifting left by N multiplies by 2^N.
3. **srl (shift right logical):** shifts 0s into the most significant bits; used for unsigned values.
4. **sra (shift right arithmetic):** copies the sign bit into the new most significant bits; divides a signed value by 2^N.
5. Shifts are **R-type**: the value in rt is shifted by shamt and the result is placed in rd.
6. **Example:** 10010000 → srl by 2 = 00100100; sra by 2 = 11100100; sll $t0,$s1,4 on 1111 0011 0000 0000 0000 0010 1010 1000 gives 0011 0000 0000 0000 0010 1010 1000 0000.
7. **Variable shifts** take the amount from a register: sllv rd, rt, rs; rt holds the value and the **5 least significant bits of rs** give the shift amount.
8. For variable shifts the order of rt and rs is reversed compared with most R-type instructions and shamt is ignored (0); e.g. $s2 = 8 shifts by 8.

Q4. [8 Marks] Explain the stored program concept and the role of the Program Counter with an example.

Answer (8 points):

1. Both 32-bit **instructions and data are stored in memory**.
2. The **only difference between two applications is the sequence of instructions** stored.
3. To run a new program there is **no rewiring** – the new program is simply stored in memory.
4. The processor **fetches** instructions from memory in sequence.
5. The processor then **performs the specified operation**.
6. The **Program Counter (PC)** keeps track of the current instruction by holding its address.
7. After each fetch the PC is normally incremented by 4; branch and jump instructions change it.

8. **Example:** the program starts at 0x00400000 and the PC steps through 0x00400004, 0x00400008, 0x0040000C.

```
Address      Machine code   Assembly
0x00400000   0x8C0A0020     lw   $t2, 32($0)
0x00400004   0x02328020     add  $s0, $s1, $s2
0x00400008   0x2268FFF4     addi $t0, $s3, -12
0x0040000C   0x016D4022     sub  $t0, $t3, $t5
```

Q5. [10 Marks] Encode into machine code (binary and hex): add $s0,$s1,$s2; sub $t0,$t3,$t5; addi $s0,$s1,5; addi $t0,$s3,-12; lw $t2,32($0); sw $s1,4($t1).

Answer (10 points):

1. **Register numbers:** $0=0, $t0=8, $t1=9, $t2=10, $t3=11, $t5=13, $s0=16, $s1=17, $s2=18, $s3=19.
2. **R-type format:** op | rs | rt | rd | shamt | funct, with assembly written as op rd, rs, rt.
3. **add $s0,$s1,$s2** → 0 | 17 | 18 | 16 | 0 | 32 = **0x02328020**.
   ```
   000000 10001 10010 10000 00000 100000
   ```
4. **sub $t0,$t3,$t5** → 0 | 11 | 13 | 8 | 0 | 34 = **0x016D4022**.
   ```
   000000 01011 01101 01000 00000 100010
   ```
5. **I-type format:** op(6) | rs(5) | rt(5) | imm(16).
6. **addi $s0,$s1,5** → 8 | 17 | 16 | 5 = **0x22300005**.
   ```
   001000 10001 10000 0000 0000 0000 0101
   ```
7. **addi $t0,$s3,-12** → 8 | 19 | 8 | -12 = **0x2268FFF4** (-12 = 0xFFF4 in 16-bit two's complement).
   ```
   001000 10011 01000 1111 1111 1111 0100
   ```
8. **lw $t2,32($0)** → 35 | 0 | 10 | 32 = **0x8C0A0020**.
   ```
   100011 00000 01010 0000 0000 0010 0000
   ```
9. **sw $s1,4($t1)** → 43 | 9 | 17 | 4 = **0xAD310004**.
   ```
   101011 01001 10001 0000 0000 0000 0100
   ```
10. **Register order:** in R-type the destination is written first in assembly but stored in rd; in I-type assembly lists rt first, yet machine code places rs before rt; in lw/sw, rs is the base register and imm is the byte offset.

Q6. [10 Marks] Explain the strategy for interpreting machine code. Decode 0x2237FFF1 and 0x02F34022 and explain how a 32-bit constant is loaded into a register.

Answer (10 points):

1. **Step 1:** read the opcode (top 6 bits) – it tells how to parse the remaining bits.
2. **Step 2:** if the opcode is all 0's the instruction is **R-type** and the funct field gives the operation.
3. **Step 3:** otherwise the opcode itself identifies the I-type or J-type instruction.
4. **Step 4:** convert register numbers to register names.
5. **Step 5:** interpret the immediate as signed or unsigned as the instruction requires (e.g. andi/ori/xori zero-extend).
6. **Decode 0x2237FFF1** = 001000 10001 10111 1111111111110001.

Field	Value	Meaning
op	001000	addi
rs	10001	$s1
rt	10111	$s7

Field	Value	Meaning
imm	1111…0001	-15

7. Result: **addi $s7, $s1, -15**.
8. **Decode 0x02F34022** = 000000 10111 10011 01000 00000 100010: opcode 0 → R-type, funct 100010 = sub.
9. Fields: rs = 23 ($s7), rt = 19 ($s3), rd = 8 ($t0) → **sub $t0, $s7, $s3**.
10. **32-bit constants:** addi handles only 16 bits, so **lui** loads the upper 16 bits and **ori** inserts the lower 16 bits (e.g. 0x6D5E4F3C).

```
lui $s0, 0x6D5E          # $s0 = 0x6D5E0000
ori $s0, $s0, 0x4F3C     # $s0 = 0x6D5E4F3C
```

4. Programming (MIPS)

Q1. [5 Marks] Explain the branch and jump instructions beq, bne, j, jr and jal.

Answer (5 points):

1. **beq rs, rt, label** branches to the label if the two registers are equal; otherwise execution continues sequentially (a label ends with a colon).
2. **bne rs, rt, label** branches only if the registers are **not equal**.
3. **j label** is an unconditional jump; the instructions in between are skipped (used for loops and skipping blocks).
4. **jr rs** (R-type) jumps to the address held in a register, decided at run time; used for returns (jr $ra).
5. **jal label** jumps to a function and saves the return address (PC+4) in $ra; e.g. jal at 0x00400200 gives $ra = 0x00400204.

Q2. [5 Marks] How are multiplication, division and 32-bit constants handled in MIPS? Give examples.

Answer (5 points):

1. Multiplying two 32-bit numbers gives a 64-bit product stored in **HI (upper 32 bits) and LO (lower 32 bits)**: mult $s0,$s1.
2. For div $s0,$s1, **LO holds the quotient and HI holds the remainder**.
3. Results are moved to general registers using **mflo and mfhi**; mul $s1,$s2,$s3 gives a 32-bit result directly.
4. **Example:** $s0 = 20, $s1 = 6 → div gives mflo $t0 = 3 (quotient) and mfhi $t1 = 2 (remainder).
5. A **32-bit constant** needs lui followed by ori (addi handles only 16 bits).

```
# int a = 0x6D5E4F3C;
lui $s0, 0x6D5E
ori $s0, $s0, 0x4F3C
```

Q3. [8 Marks] Explain how if and if/else statements are implemented in MIPS. Why is the opposite condition tested?

Answer (8 points):

1. An **if** statement executes a block only when the condition is true; it is implemented using a conditional branch and labels.
2. Example: if (i == j) f = g + h; f = f – i;

```
bne $s3, $s4, L1     # skip if i != j
add $s0, $s1, $s2    # f = g + h
L1: sub $s0, $s0, $s3 # f = f - i
```

3. If i != j the branch is taken to L1 and the if-block is skipped; if i == j it is not taken and f = g + h executes.
4. **Opposite condition:** the assembly tests the opposite so that control can skip the block when the condition is false, giving simpler and more efficient code.
5. An **if/else** statement needs a conditional branch to choose the block and an **unconditional jump** to skip the other block.
6. Example: if (i == j) f = g + h; else f = f – i;

```
bne $s3, $s4, L1
add $s0, $s1, $s2
j   done              # skip else
L1: sub $s0, $s0, $s3
done:
```

7. If i == j the if-block runs and **j done** skips the else block; if i != j control goes to L1; both paths continue at **done**.

8. A **switch/case** is implemented as a series of conditional tests (nested if/else) with a default block.

Q4. [8 Marks] Explain arrays in MIPS: address calculation, loop access and byte (character) arrays.

Answer (8 points):

1. An array stores same-type elements in sequential memory; index starts at 0 and array[0] is at the **base address**.
2. For an int array: **Address of array[i] = Base + 4i** (each element is 4 bytes).
3. Example: with base 0x10007000, array[1] = 0x10007004 and array[2] = 0x10007008.
4. The base is loaded with **lui $s0,0x1000; ori $s0,$s0,0x7000**, then elements are accessed as lw $t1, 4($s0).
5. For a large array a loop computes the address: i×4 is obtained with sll by 2.

```
# array[i] = array[i] * 8 ; $s0=base, $s1=i
sll $t0, $s1, 2      # i*4
add $t0, $t0, $s0    # address of array[i]
lw  $t1, 0($t0)
sll $t1, $t1, 3      # x 8
sw  $t1, 0($t0)
```

6. **Byte arrays** store characters in one byte: **lb** (sign-extend), **lbu** (zero-extend) and **sb** (store least significant byte).
7. For a byte array, **Address of chararray[i] = Base + i** (offset i, not 4i).
8. **Lowercase → uppercase:** subtract 32 (0x20) since 'a' = 0x61 and 'A' = 0x41.

```
addi $s1,$0,0 ; addi $t0,$0,10
loop: beq $t0,$s1,done
      add  $t1,$s1,$s0
      lb   $t2,0($t1)
      addi $t2,$t2,-32
      sb   $t2,0($t1)
      addi $s1,$s1,1
      j    loop
done:
```

Q5. [10 Marks] Explain the implementation of while and for loops in MIPS with examples, and the use of slt for magnitude comparison.

Answer (10 points):

1. A **while** loop repeats a block while the condition is true; the condition is tested before every iteration.
2. In MIPS a conditional branch exits the loop and an unconditional jump repeats it.

```
# while (pow != 128) { pow = pow*2; x = x+1; }
while: beq $s0, $t0, done   # $t0=128
       sll $s0, $s0, 1      # pow = pow*2
       addi $s1, $s1, 1     # x++
       j   while
done:
```

3. The C test is pow != 128, but the assembly tests the opposite (pow == 128) to leave the loop.
4. A **for** loop has initialisation, condition, statement and update; flow: Initialization → Condition → Statement → Update → Condition … → Exit.
5. Example: for (i = 0; i != 10; i++) sum = sum + i;

```
addi $s1,$0,0       # sum = 0
add  $s0,$0,$0      # i = 0
addi $t0,$0,10      # limit
for: beq $s0,$t0,done
     add  $s1,$s1,$s0
     addi $s0,$s0,1
```

```
     j    for
done:
```

6. The loop exits when i == 10 and the final value is sum = 0+1+…+9 = **45**.
7. **slt $t1,$s0,$s1** sets $t1 = 1 if $s0 < $s1, else 0 (e.g. 5 < 8 gives 1); beq/bne only test equality.
8. slt combined with beq or bne implements <, >, ≤ and ≥ conditions.
9. Example: for (i = 1; i < 101; i = i*2) sum = sum + i;

```
addi $s1,$0,0 ; addi $s0,$0,1 ; addi $t0,$0,101
loop: slt $t1,$s0,$t0    # i < 101 ?
      beq $t1,$0,done    # exit if i >= 101
      add $s1,$s1,$s0
      sll $s0,$s0,1      # i = i*2
      j   loop
done:
```

10. The values added are 1+2+4+8+16+32+64 = **127**.

Q6. [10 Marks] Explain function calls in MIPS – arguments, return values, the stack, preserved registers – with diffofsums, and extend it to recursive factorial.

Answer (10 points):

1. A function receives arguments in **$a0–$a3** and returns its result in **$v0–$v1**; more than four arguments go on the stack.
2. The caller uses **jal** (saves PC+4 in $ra) and the callee returns with **jr $ra**.
3. **diffofsums(f,g,h,i) = (f+g)–(h+i)** – caller code:

```
addi $a0,$0,2 ; addi $a1,$0,3 ; addi $a2,$0,4 ; addi $a3,$0,5
jal  diffofsums
add  $s0,$v0,$0        # result
```

4. Callee code (saving only $s0 – a 4-byte frame):

```
diffofsums:
  addi $sp,$sp,-4
  sw   $s0,0($sp)
  add  $t0,$a0,$a1
  add  $t1,$a2,$a3
  sub  $s0,$t0,$t1
  add  $v0,$s0,$0
  lw   $s0,0($sp)
  addi $sp,$sp,4
  jr   $ra
```

5. Result: (2+3) – (4+5) = 5 – 9 = **–4** returned in $v0.
6. The **stack** is a LIFO structure that grows toward lower addresses; $sp points to the top (addi $sp,$sp,-8 changes 0x7FFFFFFC to 0x7FFFFFF4).
7. Steps: allocate stack space, store registers, execute, restore registers, deallocate space; the allocated space is the **stack frame**.
8. **Preserved:** $s0–$s7, $ra, $sp (callee must save/restore); **nonpreserved:** $t0–$t9, $a0–$a3, $v0–$v1.
9. A **recursive function** is a nonleaf function that calls itself; each call needs its own frame to preserve $a0 and $ra (8 bytes).
10. **factorial(n) = n × factorial(n–1)**, base case factorial(1) = 1; for n = 3 the calls go 3→2→1 and returns give 1 → 2 → **6**.

```
factorial:
  addi $sp,$sp,-8
  sw $a0,4($sp) ; sw $ra,0($sp)
  addi $t0,$0,2
  slt  $t0,$a0,$t0
  beq  $t0,$0,else
```

```
  addi $v0,$0,1
  addi $sp,$sp,8
  jr   $ra
else: addi $a0,$a0,-1
  jal  factorial
  lw $ra,0($sp) ; lw $a0,4($sp)
  addi $sp,$sp,8
  mul  $v0,$a0,$v0
  jr   $ra
```

5. Addressing Modes

Q1. [5 Marks] What is an addressing mode? Why are addressing modes important? List the MIPS addressing modes.

Answer (5 points):

1. An **addressing mode** specifies how the CPU locates the operand(s) needed to execute an instruction.
2. Operands may be in **registers, memory locations, or inside the instruction** itself.
3. Addressing modes determine where operands are located and influence **execution speed and memory usage**.
4. They give flexibility for constants, variables, arrays, loops and function calls, and reduce instruction size.
5. **MIPS modes:** register-only, immediate, base, PC-relative and pseudo-direct.

Q2. [5 Marks] Explain register-only and immediate addressing with examples.

Answer (5 points):

1. **Register-only:** all source and destination operands are CPU registers; used by all R-type instructions.
2. No memory access is needed, so it is the **fastest** addressing mode; e.g. add $s0,$t2,$t3 → $s0 = $t2 + $t3.
3. Register $0 is hard-wired to 0, so sub $t8,$s1,$0 copies $s1 into $t8.
4. **Immediate:** a 16-bit constant is encoded in the instruction along with a register operand; used by I-type instructions addi, ori, andi, lui.
5. No memory access is required for the constant; e.g. addi $s4,$t5,-73 → $s4 = $t5 – 73 and ori $t3,$t7,0xFF → $t3 = $t7 OR 0x00FF.

Q3. [8 Marks] Explain base addressing with the effective address formula and examples.

Answer (8 points):

1. Base addressing is used to access **data in memory**.
2. It is used by the **lw and sw** instructions.
3. **Effective Address = Base Register + Sign-Extended 16-bit Offset.**
4. The processor first computes the effective address and then reads (lw) or writes (sw) memory.
5. The offset may be positive or negative, so locations before or after the base can be accessed.
6. **lw $s4, 72($0):** EA = $0 + 72 = 72; the word at address 72 is loaded into $s4.
7. **sw $t2, -25($t1):** EA = $t1 – 25; the contents of $t2 are stored there.
8. Applications: arrays, stack frames, structures and dynamically allocated memory, all using one instruction format.

Q4. [8 Marks] Explain PC-relative addressing with the branch target formula. Compute the offset for beq $t0,$0,else at 0x10 when else is at 0x20.

Answer (8 points):

1. PC-relative addressing is used by the conditional branches **beq and bne**.
2. The instruction stores a **16-bit signed offset** relative to the PC instead of the full destination address.
3. **BTA = (PC + 4) + (Sign-Extended Offset × 4).**
4. Advantage: shorter instruction and both **forward and backward** branches are possible.
5. In the example, beq is at PC = 0x10, so PC + 4 = 0x14 and the target else = 0x20.
6. Distance = (0x20 – 0x14)/4 = **3 instructions**, so the immediate is 3.
7. Encoding: op = 4 (beq), rs = $t0 = 8, rt = $0 = 0, imm = 3 → beq $t0,$0,3.

8. The immediate stores the **number of instructions** to skip; hardware sign-extends it, multiplies by 4 and adds it to PC+4 (0x14 + 12 = 0x20).

Q5. [10 Marks] Explain pseudo-direct addressing used by j and jal. Derive the target address and machine code for j sum located at 0x0040005C with sum at 0x004000A0.

Answer (10 points):

1. Pseudo-direct addressing is used by the jump instructions **j and jal**.
2. They are J-type and hold only a **26-bit address field** instead of the full 32-bit target.
3. **JTA = { (PC+4)[31:28], Address[25:0], 00 }.**
4. It is called **pseudo**-direct because only part of the destination address is stored in the instruction.
5. Jumps are possible anywhere within the same **256 MB** region (same upper 4 bits of PC+4).
6. **Example:** the JTA is 0x004000A0.

```
JTA = 0000 0000 0100 0000 0000 0000 1010 0000
```

7. Dropping the last two bits (shift right by 2) gives the 26-bit field **0x0100028**.
8. The opcode for j is 000010.
9. Machine code = 000010 | 00000100000000000000101000 = **0x08100028**.

```
0000 1000 0001 0000 0000 0000 0010 1000 = 0x08100028
```

10. **Why shift by 2:** every instruction is 4 bytes and word-aligned so the last two address bits are always 00; they are omitted and restored by hardware. Unlike PC-relative, an address part (not an offset) is stored.

Q6. [10 Marks] Compare the five MIPS addressing modes with the instruction types that use them, formulas and examples.

Answer (10 points):

1. **Register-only:** operands are in registers; used by R-type instructions.
2. It is the fastest mode and needs no memory access; e.g. add $s0,$t2,$t3.
3. **Immediate:** one operand is a 16-bit constant in the instruction; used by addi, ori, andi, lui.
4. It avoids fetching the constant from memory; e.g. addi $s0,$s0,10.
5. **Base:** operand is in memory; used by lw and sw; EA = Base + SignExt(offset).
6. It is ideal for arrays, stack frames and structures; e.g. lw $t0, 8($sp).
7. **PC-relative:** used by beq and bne; BTA = (PC+4) + (offset × 4).
8. It keeps branches short and allows forward and backward branching; e.g. beq $t0,$0,Loop.
9. **Pseudo-direct:** used by j and jal; JTA = {(PC+4)[31:28], addr26, 00}.
10. It reaches any address within the same 256 MB region using only 26 bits; e.g. j main.

Mode	Used by	Formula / operand
Register-only	R-type	registers
Immediate	addi, ori, andi, lui	constant in instruction
Base	lw, sw	EA = Base + offset
PC-relative	beq, bne	PC+4 + offset×4
Pseudo-direct	j, jal	{(PC+4)[31:28], addr, 00}

6. Parallelism

Q1. [5 Marks] Define latency and throughput. Calculate both for Ben baking cookies (rolling 5 min, baking 15 min, no overlap).

Answer (5 points):

1. **Latency** is the time required for one task (token) to travel from input to output.
2. **Throughput** is the number of tasks completed per unit time.
3. For a non-parallel system, **Throughput = 1 / Latency**.
4. **Cookie latency** = 5 + 15 = **20 minutes** per tray.
5. **Cookie throughput** = 60 / 20 = **3 trays per hour**, because tasks do not overlap.

Q2. [5 Marks] Differentiate spatial parallelism and temporal parallelism.

Answer (5 points):

1. **Spatial parallelism** improves performance by adding multiple copies of hardware, each doing a different task simultaneously.
2. **Temporal parallelism (pipelining)** divides a task into stages so that different tasks occupy different stages at the same time, like an assembly line.
3. **Examples:** spatial – multiple processors, ALUs, multicore CPUs; temporal – Fetch → Decode → Execute → Memory → Write back.
4. **Cost:** spatial has higher hardware cost; temporal reuses the same hardware and is more cost-effective.
5. **Throughput:** N units give N/L in spatial parallelism and N equal stages give about N/L in temporal parallelism.

Point	Spatial	Temporal
Idea	Duplicate hardware	Overlap stages
Cost	High	Lower
Example	Multicore	Pipelining

Q3. [8 Marks] Solve the cookie parallelism problem for spatial, temporal and combined parallelism, giving latency and throughput.

Answer (8 points):

1. **Base case:** latency 20 min and throughput 3 trays/hour (no overlap).
2. **Spatial:** Alyssa helps and both bake simultaneously.
3. Spatial throughput = 3 + 3 = **6 trays/hour**.
4. **Temporal:** Ben buys a second tray and prepares tray 2 while tray 1 bakes.
5. One tray enters the oven every 15 min, so temporal throughput = 60/15 = **4 trays/hour**.
6. **Spatial + temporal:** two people with two trays give **8 trays/hour**.
7. In all three cases **latency stays 20 minutes**.
8. **Conclusion:** parallelism improves throughput but does not reduce latency.

Q4. [8 Marks] What are pipeline hazards? Explain their types and the limitations of pipelining.

Answer (8 points):

1. **Pipeline hazards** are conditions that prevent instructions from flowing smoothly through the pipeline.
2. They introduce **stalls (bubbles)** that reduce throughput.
3. **Data hazard:** an instruction needs data still being produced by an earlier instruction and must wait.

4. **Control hazard:** caused by branch/jump; wrong instructions may be fetched until the branch decision is known.
5. **Structural hazard:** two instructions need the same hardware resource at the same time.
6. **Limitation:** pipeline registers and control logic add complexity.
7. **Limitation:** unequal stages reduce efficiency because the longest stage sets the clock period.
8. **Limitation:** spatial parallelism needs additional hardware, and hazards reduce performance.

Q5. [10 Marks] A circuit has clock-to-Q delay 0.3 ns, logic blocks of 3, 2 and 4 ns and setup time 0.2 ns. Find cycle time, latency and throughput without pipelining and with a two-stage pipeline. Also solve a three-stage pipeline with stage delays 3.5, 2.5 and 4.5 ns.

Answer (10 points):

1. **Without pipeline:** Tc = 0.3 + 3 + 2 + 4 + 0.2 = **9.5 ns**.
2. Latency = **9.5 ns**.
3. Throughput = 1/9.5 ns ≈ **105 MHz**.
4. **Two-stage pipeline** (register after block 2): Stage 1 = 0.3 + 3 + 2 + 0.2 = **5.5 ns**.
5. Stage 2 = 0.3 + 4 + 0.2 = **4.5 ns**.
6. The clock must suit the slowest stage, so Tc = max(5.5, 4.5) = **5.5 ns**.
7. Throughput = 1/5.5 ns ≈ **182 MHz**.
8. Latency = 2 × 5.5 = **11 ns** – about the same, so pipelining raises throughput, not latency.
9. **Three-stage pipeline** (3.5, 2.5, 4.5 ns): Tc = max = **4.5 ns**; latency = 3 × 4.5 = **13.5 ns**.

```
Tc = max(3.5, 2.5, 4.5) = 4.5 ns
Latency = 3 x Tc = 13.5 ns
```

10. Throughput = 1/4.5 ns ≈ **222 MHz**; pipeline registers isolate consecutive stages so tokens do not interfere.

Q6. [10 Marks] Explain the need for parallelism and its mathematical model. Discuss its advantages, limitations and applications.

Answer (10 points):

1. **Need:** raising clock frequency increases power consumption and heat, so performance must come from doing more work at once.
2. **Definition:** parallelism is the simultaneous execution of multiple tasks to increase the work completed in a given period (like two chefs in a kitchen).
3. **Without parallelism:** Throughput = 1/L for a task of latency L.
4. **Spatial parallelism** with N hardware units: Throughput = N/L.
5. **Temporal parallelism** with N equal stages: Throughput = N/L.
6. **Unequal stages:** the longest stage latency L1 decides speed, Throughput = 1/L1, so more stages help only if the work is balanced.

Case	Throughput
No parallelism	1 / L
Spatial (N units)	N / L
Temporal (N equal stages)	N / L
Unequal stages	1 / L1

7. **Advantages:** higher throughput, better hardware utilisation, higher clock frequency and no duplicate hardware for pipelining.
8. **Limitations:** extra hardware (spatial), pipeline registers/control complexity, unequal stages, and data/control hazards.
9. **Applications:** multicore processors, GPUs, supercomputers, DSPs and machine-learning accelerators.

10. Modern processors combine **spatial and temporal parallelism**; parallelism mainly improves throughput while latency often stays unchanged.

7. Sequential Building Blocks (Counters and Shift Registers)

Q1. [5 Marks] Explain the N-bit binary counter. How many states does it have? Show the 3-bit sequence.

Answer (5 points):

1. An N-bit binary counter is a sequential circuit with a clock input, a reset input and an N-bit output Q.
2. It is built from an **adder (adds 1) and a resettable register**.
3. Reset sets Q = 0; after reset is released, Q becomes Q + 1 on every rising clock edge and wraps to 0 after the maximum.
4. An N-bit counter has **2^N states** (0 to 2^N – 1); an 8-bit counter counts 0 to 255.
5. 3-bit sequence: 000 → 001 → 010 → 011 → 100 → 101 → 110 → 111 → 000; counters are used for timing, event counting and frequency division.

```
000 -> 001 -> 010 -> 011 -> 100 -> 101 -> 110 -> 111 -> 000
```

Q2. [5 Marks] Differentiate asynchronous (ripple) and synchronous counters.

Answer (5 points):

1. **Clocking:** in an asynchronous counter only the first flip-flop gets the main clock and each next one is clocked by the previous output; a synchronous counter uses one common clock for all flip-flops.
2. **Output change:** asynchronous outputs change one after another (ripple); synchronous outputs change in parallel.
3. **Delay:** asynchronous has cumulative propagation delay; synchronous has none.
4. **Speed:** synchronous counters work at higher frequencies; asynchronous ones are slower.
5. **Other names and design:** asynchronous = serial/ripple counter and is simple; synchronous = parallel counter and needs extra combinational logic.

Q3. [8 Marks] Explain the working of a 4-bit asynchronous ripple counter with its count table. Describe the decade counter.

Answer (8 points):

1. In a ripple counter only the first flip-flop gets the clock; the **output of each flip-flop clocks the next**.
2. **Q0 toggles on every clock pulse.**
3. **Q1 toggles every 2 pulses, Q2 every 4 and Q3 every 8**, giving the binary sequence 0 to 15.
4. Each stage divides frequency by 2 (Q0 = f/2, Q1 = f/4); one FF is a MOD-2 and two FFs form a MOD-4 counter.
5. Count table: reset 0000, pulse 1 → 0001, 2 → 0010, 7 → 0111, 8 → 1000, 15 → 1111 and pulse 16 wraps to 0000.

Clock	Q3 Q2 Q1 Q0	Decimal
Reset	0000	0
1	0001	1
2	0010	2
7	0111	7
8	1000	8
15	1111	15
16	0000	0 (wraps)

6. The 4-bit ripple counter has 16 states (MOD-16).

7. **Decade counter:** counts ten states (0–9) and then resets.
8. A **NAND gate on Q3 and Q1** (binary 1010 = 10) drives the clear input, so the counter resets at 10 and restarts from 0.

Q4. [8 Marks] Explain ring counter and Johnson counter with their sequences. Compare them.

Answer (8 points):

1. A **ring counter** is a shift-register counter whose last flip-flop output is fed back to the first.
2. A single 1 circulates: **1000 → 0100 → 0010 → 0001 → 1000**; n flip-flops give n states (MOD-n).
3. It must be initialised with a single 1; **0000 is an invalid/stuck state**.
4. Ring counters are simple and fast and give decoded outputs directly, but need n flip-flops for n states; used for timing, sequence generation and stepper motor control.
5. A **Johnson (twisted-ring/switch-tail) counter** feeds the **complement** of the last flip-flop back to the first (D0 = Q3').
6. 4-bit sequence: **0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000**.
7. For n flip-flops a Johnson counter has **2n states** (8 states for 4 bits).
8. **Comparison:** ring – n states, direct decoding; Johnson – 2n states for the same number of flip-flops, needs simple decoding.

Point	Ring	Johnson
Feedback	Q of last FF	Q' of last FF
States (n FFs)	n	2n

Q5. [10 Marks] Explain the shift register, shift register with parallel load and its HDL. Describe scan chains and how a 32-bit counter is tested in 65 cycles.

Answer (10 points):

1. A **shift register** is a chain of N flip-flops sharing one clock.
2. Each flip-flop's output feeds the next flip-flop's input; a reset may clear all of them.
3. On each clock edge a new bit enters at **Sin** and every stored bit moves forward; the oldest bit leaves at **Sout**.
4. After N cycles the N bits are available on the parallel outputs, so it converts **serial to parallel**.
5. **Parallel load:** a D input and a Load control with a mux before each flip-flop; Load = 1 loads D in one cycle, Load = 0 shifts normally.
6. **HDL:** reset has highest priority, then load, otherwise shift.

```
always @(posedge clk or posedge rst)
  if (rst)       q <= 0;
  else if (load) q <= data_in;
  else           q <= {q[N-2:0], sin};
assign sout = q[N-1];
```

7. **Testing problem:** sequential circuits depend on internal state; checking bit 31 of a 32-bit counter would need ~2 billion clocks and there are too many flip-flops for individual test pins.
8. **Scan chain:** all flip-flops are connected into one long shift register using **scannable flip-flops** (a mux built into each FF).
9. In **normal mode** each FF loads from its regular D input; in **test mode** data is shifted serially in/out so internal state can be set or read.
10. **32-bit counter test:** shift in 0111…1 (32 cycles), run 1 normal clock (1 cycle), shift out and check 1000…0 (32 cycles) – total **65 cycles**.

```
32 (shift in) + 1 (count) + 32 (shift out) = 65 cycles
```

Q6. [10 Marks] Design a 3-bit synchronous up counter using T flip-flops and a 2-bit synchronous up counter using JK flip-flops. Also design the 32-bit counters of Exercises 5.43 and 5.44.

Answer (10 points):

1. **Design steps:** decide the number of flip-flops, write the excitation table, draw the state diagram and circuit excitation table, solve using K-maps, draw the logic diagram.
2. **T flip-flop excitation:** Qn→Qn+1: 0→0: T=0; 0→1: T=1; 1→0: T=1; 1→1: T=0 (T=1 toggles the FF).
3. **3-bit up counter** uses 3 T flip-flops with one common clock and states 000→001→… →111→000.
4. K-maps give **T0 = 1, T1 = Q0, T2 = Q1·Q0**.

```
T0 = 1
T1 = Q0
T2 = Q1 . Q0
```

5. A bit toggles only when all lower bits are 1.
6. **2-bit up counter with JK FFs** uses 2 flip-flops and the JK excitation table.
7. K-maps give **J0 = K0 = 1** and **J1 = K1 = Q0**.

```
J0 = K0 = 1
J1 = K1 = Q0
```

8. **Exercise 5.43 (32-bit up/down counter):** Q+ = 0 if Reset, Q+1 if Up = 1, Q–1 if Up = 0; use a 32-bit register, adder/subtractor and mux controlled by Up.
9. **Exercise 5.44 (adds 4 each clock):** Q+ = Q + 4 with Reset → 0; use a 32-bit register, a 32-bit adder and constant 4.
10. In both exercises **Reset is synchronous** – it overrides the next-state value at the clock edge.

8. Memory Arrays

Q1. [5 Marks] Explain the organisation of a memory array. Find the size of a memory with 10 address bits and 32-bit data.

Answer (5 points):

1. A memory array is a 2-D array of 1-bit cells; each row is a word and each column is a bit.
2. N address bits select one of **2^N words**, each of M bits, so **size = 2^N × M bits**.
3. Each bit cell connects to a **wordline** (selects the row) and a **bitline** (carries data).
4. A **decoder** asserts one wordline for each address; memories are specified by size and by number/type of ports.
5. **Example:** 10 address bits and 32-bit data → 2^10 = 1024 words → 1024 × 32 = **32 Kb**.

```
N = 10 -> 2^10 = 1024 words
Size = 1024 x 32 = 32768 bits = 32 Kb
```

Q2. [5 Marks] Differentiate RAM and ROM.

Answer (5 points):

1. **Full form:** RAM = Random Access Memory; ROM = Read Only Memory.
2. **Volatility:** RAM is volatile (loses data when power is off); ROM is nonvolatile.
3. **Operations:** RAM is read/write memory; ROM is historically read-only (modern ROMs can be written in various ways).
4. **Types:** RAM – DRAM and SRAM; ROM – Mask ROM, PROM, EPROM, EEPROM and Flash.
5. **Access:** both can be accessed randomly.

Q3. [8 Marks] Compare DRAM, SRAM and flip-flop memories. Why is DRAM slower? Explain SDRAM and DDR SDRAM.

Answer (8 points):

1. **DRAM** stores each bit as charge on a capacitor (about 1 transistor per bit), so it is very dense.
2. DRAM charge leaks, so **periodic refresh** is needed, and a read destroys the value so it is restored.
3. **SRAM** stores a bit in **cross-coupled inverters**; no refresh is needed and noise is corrected.
4. **Area:** flip-flop ≈ 20 transistors/bit, SRAM = 6, DRAM = 1.
5. **Latency:** flip-flop fastest, SRAM medium, DRAM slowest.

Memory	Transistors/bit	Latency
Flip-flop	~20	Fast
SRAM	6	Medium
DRAM	1	Slow

6. **Why DRAM is slower:** charge moves from the capacitor to a bitline that is not actively driven, and refresh lowers throughput.
7. **SDRAM** (Synchronous DRAM) uses a clock and pipelines memory accesses.
8. **DDR SDRAM** transfers data on both rising and falling clock edges, doubling throughput for a given clock speed.

Q4. [8 Marks] Explain the register file with the 32 × 32 three-ported example.

Answer (8 points):

1. A **register file** is a group of registers storing temporary variables in a digital system.
2. It is usually a small **multi-ported SRAM array**, more compact than flip-flop registers.

3. The 32 × 32 register file has 32 registers of 32 bits each.
4. It has **two read ports** (A1 → RD1, A2 → RD2).
5. It has **one write port** (A3, WD3), controlled by CLK and the write enable.
6. Each address is **5 bits** because 2^5 = 32.
7. Two registers can be **read and one written in the same cycle** (e.g. two source registers and one destination of an instruction).
8. Total storage = 32 × 32 = **1024 bits**.

Q5. [10 Marks] Explain ROM: bit-cell working, dot notation and the types of ROM (Mask, PROM, EPROM, EEPROM, Flash). Why is ROM nonvolatile?

Answer (10 points):

1. A ROM stores each bit by the **presence or absence of a transistor**.
2. The bitline is weakly pulled HIGH and the wordline is turned ON.
3. If a transistor is **present** it pulls the bitline LOW, so the stored bit is 0; if **absent** the bitline stays HIGH and the bit is 1.
4. ROM is **nonvolatile** because its bit cell is combinational and has no state that is lost when power is removed.
5. **Dot notation:** rows are wordlines (addresses), columns are bitlines; a dot at an intersection means stored 1 (e.g. a 4×3 ROM stores 12 bits).
6. **Mask ROM:** contents fixed during manufacturing; cannot be changed.
7. **PROM:** transistors initially present; programming blows fuses with high voltage, so it is one-time programmable.
8. **EPROM:** uses a floating-gate transistor; programmed with high voltage and erased with intense **UV light**.
9. **EEPROM:** programmed and erased electrically, individual cells can be erased.
10. **Flash:** like EEPROM but erases larger blocks, needing fewer erase circuits and so cheaper.

Type	Programmable	Erasable	Method
Mask ROM	At manufacture	No	—
PROM	Once	No	—
EPROM	Yes	Yes	UV light
EEPROM	Yes	Yes	Electrical
Flash	Yes	Yes	Electrical, block-wise

Q6. [10 Marks] Explain how memory arrays implement combinational logic (LUT). Implement Y = AB using a LUT and explain the RAM and ROM HDL descriptions.

Answer (10 points):

1. A 2^N-word × M-bit memory can implement **any combinational function with N inputs and M outputs**.
2. The N inputs form the **address**; the data stored in the selected row gives the M outputs.
3. The size of the memory is **2^N × M**; the contents are the truth table.
4. A ROM can be seen as two-level logic: the address passes through a **decoder (AND gates giving all minterms)** and then **OR gates**.
5. A memory array used to perform logic is called a **Lookup Table (LUT)**.
6. **Y = AB** needs a 4 × 1 memory because there are 2^2 = 4 input combinations; it stores 0, 0, 0, 1.

A	B	Address	Y = AB
0	0	00	0
0	1	01	0

A	B	Address	Y = AB
1	0	10	0
1	1	11	1

7. **RAM HDL:** a 2^N × M RAM is declared as an array of M-bit words (N = 6, M = 32 gives 64 × 32).
```
logic [M-1:0] mem [2**N-1:0];
```

8. **RAM write** is synchronous: it occurs on the rising clock edge when we = 1 (mem[adr] <= din).
9. **RAM read** is immediate/asynchronous (assign dout = mem[adr]); RAM contents are unpredictable at power-up.
10. **ROM HDL:** contents are fixed using a case statement on the address.
```
always_comb case(adr)
  2'b00: dout <= 3'b011;  2'b01: dout <= 3'b110;
  2'b10: dout <= 3'b100;  2'b11: dout <= 3'b010;
endcase
```